LAB REPORT · 01

University of Azad Jammu and Kashmir

Department of Software Engineering

Digital Image Processing Lab

Lab Task #01

Python Setup, Fundamentals, and Imports

Installation and setup · Python fundamentals · Package imports

STUDENT NAME	Sheikh Haseeb-ur-Rehman
ROLL NUMBER	2024-SE-31
INSTRUCTOR	Engr. Fahad Nisar
SEMESTER	Fifth
SESSION	2024-28 (SE)
DATE	October 22, 2026

1 Installation and Setup

Objective: Verify the installation of Python, the `pip` package manager, and the configuration of the Visual Studio Code environment.

Version Check Commands:

```
python --version
python -m pip --version
```

Listing 1 — Commands used to verify Python and pip

Python and Pip Versions:



Figure 1 — Python and pip versions

VS Code Interpreter Selection:

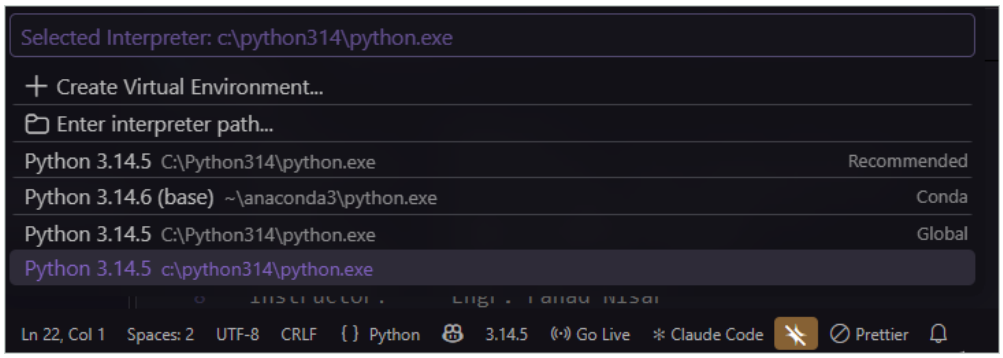


Figure 2 — VS Code interpreter selection

First Python Program (hello.py):

Code

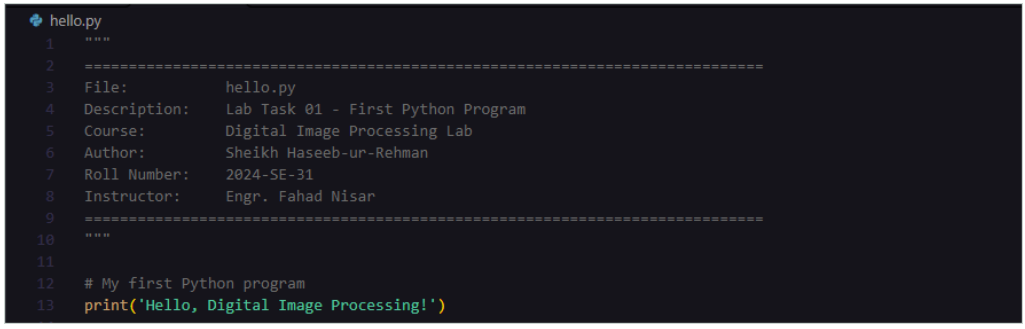


Figure 3 — hello.py code

▸ Output

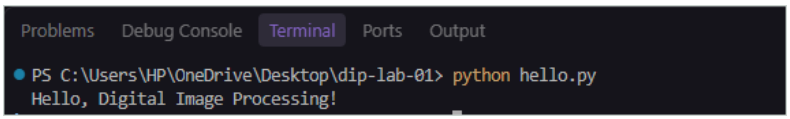


Figure 4 — hello.py terminal output

2 Python Fundamentals (basics.py)

Objective: Demonstrate proficiency in Python variables, data types, conditional statements, loops, functions, and collections.

Exercises 1 & 2: Variables, Data Types, and Conditions

■ Exercise 1

▸ Code

```python
# Exercise 1: Variables and Data Types
print_separator("Exercise 1: Variables & Data Types")
width = 640
height = 480
image_name = "sample.png"
is_color = True

print(f"Variable 'width'      : {width} \t| Type: {type(width)}")
print(f"Variable 'height'     : {height} \t| Type: {type(height)}")
print(f"Variable 'image_name' : {image_name} \t| Type: {type(image_name)}")
print(f"Variable 'is_color'   : {is_color} \t| Type: {type(is_color)}")

total_pixels = width * height
print(f"\nTotal calculated pixels (width * height): {total_pixels}")
```

Figure 5 — Exercise 1 code

▸ Output

```
--------------- Exercise 1: Variables & Data Types ---------------
Variable 'width'      : 640     | Type: <class 'int'>
Variable 'height'     : 480     | Type: <class 'int'>
Variable 'image_name' : sample.png      | Type: <class 'str'>
Variable 'is_color'   : True    | Type: <class 'bool'>

Total calculated pixels (width * height): 307200
```

Figure 6 — Exercise 1 output

Exercise 2

Code

```python
# Exercise 2: if, elif, and else
print_separator("Exercise 2: if, elif, and else")
test_values = [-1, 0, 127, 128, 255, 256]

for val in test_values:
    if val < 0 or val > 255:
        print(f"Intensity {val:>3} : Invalid")
    elif val >= 128:
        print(f"Intensity {val:>3} : Bright pixel")
    else:
        print(f"Intensity {val:>3} : Dark pixel")
```

Figure 7 — Exercise 2 code

Output

```
--------------- Exercise 2: if, elif, and else ---------------
Intensity  -1 : Invalid
Intensity   0 : Dark pixel
Intensity 127 : Dark pixel
Intensity 128 : Bright pixel
Intensity 255 : Bright pixel
Intensity 256 : Invalid
```

Figure 8 — Exercise 2 output

Exercises 3 & 4: Loops and Functions

Exercise 3

Code

```python
# Exercise 3: for and while Loops
print_separator("Exercise 3: for and while Loops")
print("--- For Loop Output ---")
for pixel in [20, 130, 255]:
    label = "Bright pixel" if pixel >= 128 else "Dark pixel"
    print(f"Value {pixel:>3} is a {label}")

print("\n--- While Loop Output ---")
i = 0
while i < 3:
    print(f"Counter: {i}")
    i += 1  # update the counter so the loop stops before 3
```

Figure 9 — Exercise 3 code

Output

```
--------------- Exercise 3: for and while Loops ---------------
--- For Loop Output ---
Value  20 is a Dark pixel
Value 130 is a Bright pixel
Value 255 is a Bright pixel

--- While Loop Output ---
Counter: 0
Counter: 1
Counter: 2
```

Figure 10 — Exercise 3 output

Exercise 4

Code

```python
# Exercise 4: Functions
print_separator("Exercise 4: Functions")


def is_bright(pixel, threshold=128):
    """Return True if the pixel is at least the threshold, else False."""
    return pixel >= threshold


print(f"is_bright(127)      -> {is_bright(127)}")
print(f"is_bright(128)      -> {is_bright(128)}")
print(f"is_bright(200, 210) -> {is_bright(200, 210)}")
```

Figure 11 — Exercise 4 code

Output

```
--------------- Exercise 4: Functions ---------------
is_bright(127)      -> False
is_bright(128)      -> True
is_bright(200, 210) -> False
```

Figure 12 — Exercise 4 output

Exercises 5 & 6: Collections (Lists, Tuples, Dictionaries, Sets)

Exercise 5

Code

```python
# Exercise 5: Lists and Tuples
print_separator("Exercise 5: Lists and Tuples")
pixels = [0, 128, 255]
print(f"Original list       : {pixels}")
print(f"First item          : {pixels[0]}")

pixels[1] = 150      # replace the middle item
pixels.append(64)    # add a new item at the end
print(f"Modified list       : {pixels}")
print(f"Total list length   : {len(pixels)}")

size = (640, 480)
width, height = size  # unpack the tuple
print(f"\nUnpacked Tuple      : Width = {width}, Height = {height}")

# Explanation: A list is mutable. It is stored so that its items can be
# reassigned, added or removed in place (pixels[1] = 150, pixels.append(64)).
# A tuple is immutable. It does not support item assignment, so size[0] = 100
# raises a TypeError. Its items are fixed once the tuple is created.
```

Figure 13 — Exercise 5 code

Output

```
--------------- Exercise 5: Lists and Tuples ---------------
Original list       : [0, 128, 255]
First item          : 0
Modified list       : [0, 150, 255, 64]
Total list length   : 4

Unpacked Tuple      : Width = 640, Height = 480
```

Figure 14 — Exercise 5 output

Exercise 6

Code

```python
# Exercise 6: Dictionaries and Sets
print_separator("Exercise 6: Dictionaries and Sets")
image = {"width": 640, "height": 480}
image["mode"] = "RGB"

print(f"Dictionary Width Key: {image['width']}")
print(f"Dictionary Mode Key : {image['mode']}")

labels = ["cat", "dog", "cat", "bird"]
unique_labels = set(labels)
print(f"\nOriginal labels list: {labels}")
print(f"Unique labels (set) : {unique_labels}")
print(f"Count of unique labels: {len(unique_labels)}")
```

Figure 15 — Exercise 6 code

▸ **Output**

```
--------------- Exercise 6: Dictionaries and Sets ---------------
Dictionary Width Key: 640
Dictionary Mode Key : RGB

Original labels list: ['cat', 'dog', 'cat', 'bird']
Unique labels (set) : {'dog', 'cat', 'bird'}
Count of unique labels: 3
```

Figure 16 — Exercise 6 output

Understanding Mutability: As demonstrated in Exercise 5, a list is a mutable collection: its items can be reassigned, added or removed in place (for example pixels[1] = 150 and pixels.append(64)). A tuple is immutable: it does not support item assignment, so an attempt such as size[0] = 100 raises a TypeError, and its items stay fixed once the tuple is created.

3 Package Management and Imports

Objective: Install required external libraries for digital image processing and verify their successful import.

■ Installation Commands:

```
python -m pip install numpy pillow matplotlib
python -m pip show numpy pillow matplotlib
```

Listing 2 — Installing and inspecting the required packages

■ Installed Package Details:

```
Name: matplotlib
Version: 3.11.2
Summary: Python plotting package
Home-page: https://matplotlib.org
Author: John D. Hunter, Michael Droettboom
Author-email: Unknown <matplotlib-users@python.org>
License: License agreement for matplotlib versions 1.3.0 and later

PS C:\Users\HP\OneDrive\Desktop\dip-lab-01> python -m pip show numpy pillow matplotlib | more
Name: numpy
Version: 2.5.4
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
Author: Travis E. Oliphant et al.
Author-email:
License-Expression: BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0
Location: C:\Users\HP\AppData\Roaming\Python\Python314\site-packages
Requires:
Required-by: contourpy, matplotlib, pandas
Name: pillow
Version: 13.0.0
Summary: Python Imaging Library (fork)
Home-page: https://python-pillow.org
```

Figure 17 — pip show output for numpy, pillow, matplotlib

Import Verification (lab01_imports.py):

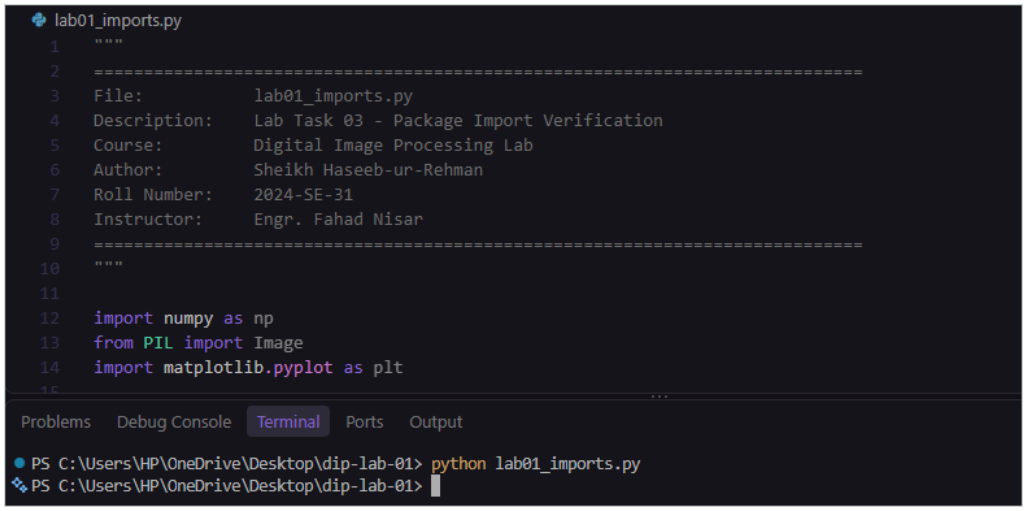


Figure 18 — lab01_imports.py code

4 Technical Summary

Python serves as the core runtime language that executes our programmatic logic, while `pip` functions as its dedicated package manager, enabling us to easily download and install external libraries like NumPy and Pillow. The `import` statements are then required to link these external libraries into our current working script, granting us access to their pre-built, specialized functions for tasks such as numerical computation, image manipulation, and data visualization.

Key Import Statements:

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

Listing 3 — Imports linking NumPy, Pillow, and Matplotlib into the script